

Analysis: Boring Numbers

Test Set 1

Simply check all the numbers from **L** to **R**. Complexity = $O((R-L) \times \log_{10}(R))$.

Test Set 2

We cannot follow the same approach as in Test Set 1 because the limits are too high. Let us calculate the number of boring numbers having exactly X digits first.

Lemma 1: There are 5 choices for digits at odd positions: $\{1,3,5,7,9\}$, and 5 choices for digits at even positions: $\{0,2,4,6,8\}$. Thus, the total number of boring numbers having exactly X digits is 5^X .

Let us calculate the number of boring numbers less than or equal to **R**. The general idea is to split $[1, R]$ into minimum number of intervals such that, all the numbers in an interval,

- Are of the same length.
- Have some(possibly none) prefix digits fixed.
- Positions following the fixed prefix can take any values from $[0,9]$.

Let $\text{length}_{\text{interval}}$ be the length of numbers in an interval. The intervals can thus be broken into two cases:

1. $\text{length}_{\text{interval}} < \text{length of } R$
2. $\text{length}_{\text{interval}} = \text{length of } R$

For example, let **R** = 3422. Number of boring numbers in $[1, R]$ equals the number of boring numbers in $[1,9] + [10, 99] + [100, 999] + [1000, 1999] + [2000, 2999] + [3000, 3099] + [3100, 3199] + [3200, 3299] + [3300, 3399] + [3400, 3409] + [3410, 3419] + [3420, 3422]$.

Case 1: This can be calculated using the fact that the total number of boring numbers having exactly X digits is 5^X .

Case 2: Suppose **R** = $d_1, d_2, d_3, \dots, d_{\text{len}}$. Intervals can be calculated by fixing some prefix of digits such that all numbers in the interval are less than **R**. For example, all numbers of the form, $d_1, \dots, d_{i-1}, a_i, x_{i+1}, x_{i+2}, \dots, x_{\text{len}}$, where $a_i < d_i$ and $0 \leq x_{i+1}, \dots, x_{\text{len}} \leq 9$.

If the fixed prefix does not obey the boring number's criteria, the number of boring number in the interval is 0. If it does, then, similar to Lemma 1, the number of boring numbers equals $5^{\text{length of suffix}}$.

Final answer = (number of boring numbers less than or equal to **R**) - (number of boring numbers less than or equal to **L** - 1).

Complexity = $O(\log_{10}(R))$.

Analysis: Friends

Test Set 1

For this test set, we can build an undirected graph, where the nodes represent people and there is an edge between two nodes if and only if the names of the respective people have a common letter. Then each query can be answered by finding a shortest path between two nodes in the graph.

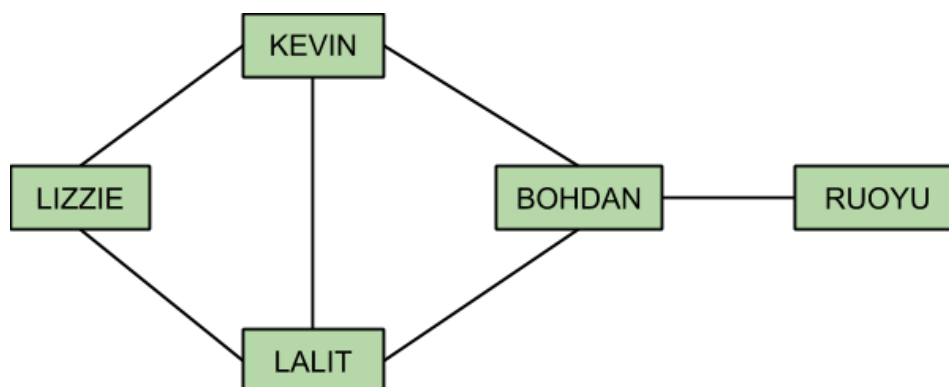
Assuming that L is the length of the longest name (recall that $L \leq 20$), it is straightforward to build the graph in $O(L^2 N^2)$ time by considering each pair of nodes and using a nested pair of loops to look for a common letter in the names. The time complexity can be reduced to $O(LN^2)$ if we mark the letters of one of the names, and then iterate over the letters of the other name to see if any of those letters has been marked. A better yet approach is to represent the letters of a name as a 32-bit integer, where the i -th bit is 1 if the i -th letter of the alphabet is present in the name, and 0 otherwise. Then testing for common letters in two names boils down to checking if the logical AND of the respective bitmasks is non-zero, and thus the graph can be built in $O(LN + N^2)$ time.

The graph may have up to $N \times (N-1) / 2$ edges, therefore, if we answer each query independently, say, using the [breadth-first search](#), the overall time complexity of answering all Q queries is $O(QN^2)$, which might be too slow for large values of Q . Even if we never compute the answer for the same query twice by memoizing the results in a two-dimensional array, the time complexity is still $O(N^4)$.

A better alternative is to pre-compute the whole distance array in $O(N^3)$ time using the [Floyd-Warshall](#) algorithm or running breadth-first search N times, once from each node. Then the individual queries can be answered in constant time and the overall time complexity becomes $O(LN + N^3 + Q)$.

Test Set 2

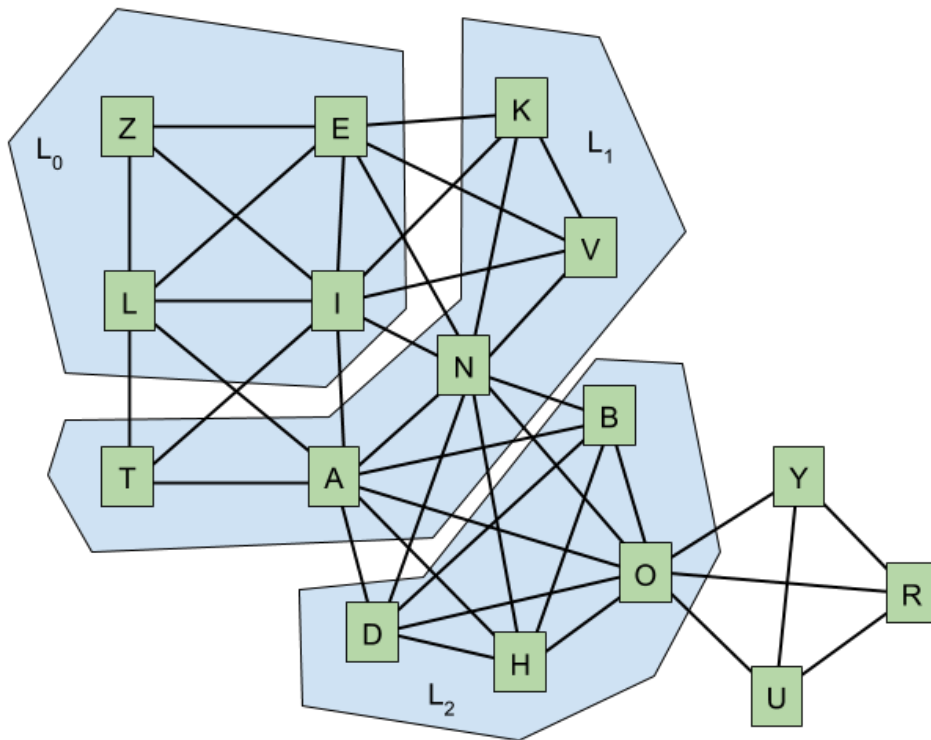
Here N can be large, so we really need an algorithm that is subquadratic in N . Let us look at the problem from a different angle. Using the first sample test case, consider the friendship graph below, and suppose that we need to find the length of the shortest friendship chain between LIZZIE and RUOYU.



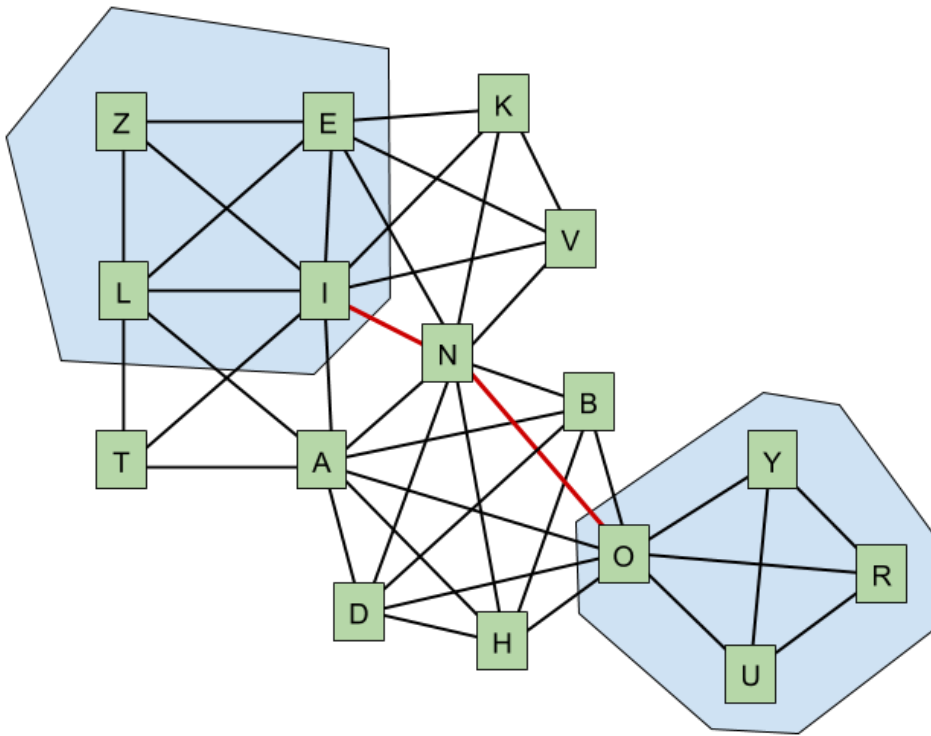
We start out with the set of letters $L_0 = \{L, I, Z, E\}$. Since the names KEVIN and LALIT have at least one letter in L_0 , we know that the shortest friendship chain to these names has length 2.

By extending our search to the names KEVIN and LALIT, we have reached the letters $L_1 = \{K, V, N, A, T\}$. Since BOHDAN has some letters in L_1 (N and A), the length of the shortest friendship chain from LIZZIE to BOHDAN is 3, and adding BOHDAN leads to a new set of letters $L_2 = \{B, O, H, D\}$. The name RUOYU has the letter O in L_2 , therefore, the shortest friendship chain to RUOYU has length 4.

The crucial observation here is that we are essentially doing a breadth-first search in a very small graph G on the set of nodes $\{A, B, C, \dots, X, Y, Z\}$, where there is an edge between a pair of letters $\{u, v\}$ if and only if the letters occur in the same name. The graph for our example is illustrated below.



The goal of the breadth-first search was to find the shortest path connecting the sets of letters $\{L, I, Z, E\}$ and $\{R, U, O, Y\}$. One of such paths is highlighted in red in the picture below. Instead of doing the breadth-first search, we could also pre-compute all-pairs shortest paths in G in constant time (yes, $O(26^3)$ is still $O(1)$), and then answer each query in $O(L^2)$ time by taking the minimum distance over all pairs of letters $\{a, b\}$, where the letter a occurs in the first name, and the letter b occurs in the other name.



We can build the graph G in $O(L^2N)$ time by considering each of the input names and creating an edge between each pair of letters in the name. Consequently, the overall time complexity of the algorithm is $O(L^2(N+Q))$.

Analysis: Retype

There are 2 possible options to complete all the **N** levels. Let answer1 and answer2 be the time taken to complete all levels for each of the options respectively. Initialize answer1 and answer2 as 0.

Test set 1

We have already reached the level **K**. In order to calculate time taken to reach level **K**, start iterating from level 1 till level **K** and increment answer1 and answer2 on each step as we take 1 unit time to complete each level.

- One of the options to complete all levels is to restart the game and complete all the levels again, starting from level 1. We can calculate the time taken for this option by iterating from level 1 till level **N** and incrementing answer1.
- The second possible option is to go back to level **S** and then complete remaining levels after picking up the sword at level **S**. We can calculate the time taken for this option by iterating from level **K** to level **S** and incrementing answer2 at each step. Now, start iterating from level **S** till level **N** and increment answer2 at each step.

The minimum time taken to complete all of the levels is the minimum value among answer1 and answer2. As we iterate each level at most twice in each possible option, the complexity of the solution is $O(N)$.

Test set 2

An observation here is that instead of simulating the levels and calculating the time taken, we can compute it in $O(1)$ time. Initially, we are at level **K**. This means that we took **K** time units to reach level **K** as completing each level takes 1 unit of time. Hence, we can increment answer1 and answer2 by **K** directly.

- For the first option, we restart the game and complete all levels again. It would take **N** time units to complete all the levels again as total number of levels are **N**. Hence, $\text{answer1} = N + K$.
- For the second option, we go back to level **S**. This would take **K - S** time units as there are **K - S** levels between level **K** and level **S**. Then we complete remaining levels after picking up sword at level **S**. This would take **N - S** time units. Hence $\text{answer2} = K + K - S + N - S$.

The minimum time required to complete all levels is the minimum value among answer1 and answer2. Note that answer2 can overflow the range of range of 32-bit signed integers. For example, in C++ answer2 may overflow the range of INT datatype. Although the minimum answer would fit in the range of INT but overflowing of answer2 can cause unexpected output. Computing both answer1 and answer2 can be done in constant time. Hence, time complexity of the solution is $O(1)$.

Analysis: Rugby

We can divide this problem into two parts: choosing where to place the line, and choosing the permutation of players on the line (in other words, which players should be on which points on the line).

Regarding the permutation of players, the first observation that we can make is that preserving the initial X-ordering of the players is an optimal strategy. That means we should place the player with the lowest X coordinate value as the first player on the line, the player with the second lowest X coordinate value as the second player on the line, and so on. If there are ties, we can break them arbitrarily.

We can prove that this greedy strategy is optimal. First let us assume that there are two players with X -coordinates X_1 and X_2 , and let us denote the X -coordinate of the start of line as $startX$, whereas $X_1 \leq startX < X_2$. If we do not preserve the players' relative position on the final solution, we have that the first player will move for $(startX + 1) - X_1$ steps, and the second player will move for $X_2 - startX$ steps, so the final solution equals $-X_1 + X_2 + 1$ steps. However, if we preserved the players' relative positions, the first player would move for $startX - X_1$ steps, and the second player would move for $X_2 - (startX + 1)$ steps, so the final solution would equal $-X_1 + X_2 - 1$ steps, which is lower than the above strategy. For the other possible values of $startX$, which are $startX < X_1$ or $X_2 \leq startX$, both permutations for the players result on the same amount of steps. Therefore, we can reach the conclusion that, for any solution, by swapping two players' order in order to preserve the players' initial relative positions, the total number of steps remains the same or decreases.

Now imagine that we have an arbitrary optimal solution S (not necessarily sorted on the players' X -coordinates). If we perform a swap, as described above, the solution does not get any worse, so it is still optimal. If we perform such a swap enough times we will reach a point in which the players are in non-descending order of their X coordinate, which is what our greedy solution looks like, and the solution is still optimal. Therefore, our greedy solution is optimal.

Regarding where to place the line, let *solutionFirstX* be the X position of the first point on the line, and let *solutionRow* be the Y position of the line.

Test Set 1

Given the constraints of the Test Set 1, we can iterate through all possible values for *solutionFirstX* and *solutionRow* and check which combination results on the minimum total number of steps. The grid is unbounded, however we can bound the possible values for *solutionFirstX* based on the initial X positions of the players, because placing the line entirely to the left of the player with the lowest X coordinate would make them all take an extra step to get there, which intuitively leads to a worse solution. The same applies on the right side. Let *minPlayerX* represent the X value of the player the most to the left, and *maxPlayerX* represent the X value of the player the most to the right. The number of possible values for *solutionFirstX* range between *minPlayerX*-($N-1$) and *maxPlayerX*. Given similar definitions for *minPlayerY* and *maxPlayerY*, the same logic applies for *solutionRow*, which ranges between *minPlayerY* and *maxPlayerY*.

To calculate what is the total number of steps for the players to reach the line at *candidateFirstX* and *candidateRow*, iterate through all players in non-descending X -coordinate order and add their [Manhattan distances](#) to their relative locations on the line. In other words, the total number

of steps equals the sum of $|X_i - (\text{candidateFirstX} + (i - 1))| + |Y_i - \text{candidateRow}|$, for every player i in non-descending order of X_i .

The complexity of given solution is $O(N \times \log N + (\text{maxPlayerX} - \text{minPlayerX} + N) \times (\text{maxPlayerY} - \text{minPlayerY} + 1) \times N)$.

Test Set 2

Given that the limits for N and the players initial positions are much greater in Test Set 2, the above mentioned solution would not be fast enough.

To divide our solution into smaller steps, we should notice that horizontal and vertical movement can be solved independently. In other words, the value *solutionRow* does not affect how many horizontal steps the players will take, as well as *solutionFirstX* does not affect how many vertical steps the players will take.

Let us start by finding the value for *solutionRow*. We can observe that most players will have to move vertically to reach *solutionRow*, either by moving up or moving down (and some will not have to move because they are initially on *solutionRow*). We want the total number of steps to be as small as possible, so a good candidate is the Y-position of the player that is positioned as the median among all players, regarding their initial Y-position.

We can prove that this is optimal as follows: first, let us start with the assumption that N is odd. If we set *solutionRow* to be equal to $Y_{\lceil N/2 \rceil}$ (the Y-position of the player in the middle), then we can see that there will be at least $\lfloor N/2 \rfloor$ players who are initially below or at the same line as the one in the middle, and at least $\lfloor N/2 \rfloor$ players who are initially above or at the same line as the one in the middle. Let us call *totalStepsBelow* the total number of steps taken by the $\lfloor N/2 \rfloor$ players who are initially below or at the same line as the one in the middle, and let us call *totalStepsAbove* the total number of steps taken by the $\lfloor N/2 \rfloor$ players who are initially above or at the same line as the one in the middle. As is, the total number of steps equals *totalStepsBelow* + *totalStepsAbove*. If we move *solutionRow* one position up, then the player in the middle and all the players that were initially below or at the same line as the one in the middle will have to take one extra step to reach *solutionRow*, and, in the best scenario, all the players that were initially above as the one in the middle will have to take one less step to reach *solutionRow*. Therefore, the total number of steps would equal $(\text{totalStepsBelow} + \lfloor N/2 \rfloor + 1) + (\text{totalStepsAbove} - \lfloor N/2 \rfloor)$, which equals *totalStepsBelow* + *totalStepsAbove* + 1, which is worse than our first guess. The same applies for moving *solutionRow* one position down. A similar logic can be used for when N is even.

Now we can use a similar approach for finding the value for *solutionFirstX*. Taking into account the fact that we want to preserve the players initial X-ordering, we can observe that for each player i , who has k_i players to his left, the optimal value for *solutionFirstX* would be $X_i - k_i$, because this way he would not have to move horizontally at all. Using the same logic described for finding *solutionRow*, we can find the optimal value for *solutionFirstX* by picking the median of all players optimal values for *solutionFirstX*.

Using these strategies we can find the value for *solutionRow* in $O(N \times \log N)$ time by sorting the players by their initial Y-coordinate and picking the median value. Similarly, we can find the value for *solutionFirstX* in $O(N \times \log N)$ time by sorting the players by their initial X-coordinate and picking the median optimal value. Finally we can calculate the total number of steps in $O(N)$ time, by using the strategy mentioned on Test Set 1 analysis. Therefore, the time complexity of the solution is $O(N \times \log N)$.